HTTP AND HTTPS PASSWORD CAPTURE USING WIRESHARK

A SUMMER INTERNSHIP REPORT

Submitted by

ELAVARASAN.S – 810023205307



DEPARTMENT OF INFORMATION TECHNOLOGY

UNIVERSITY COLLEGE OF ENGINEERING ,TIRUCHIRAPALLI
(BIT CAMPUS), ANNA UNIVERSITY, – 620 024

UNIVERSITY COLLEGE OF ENGINEERING ,TIRUCHIRAPALLI
(BIT CAMPUS), ANNA UNIVERSITY, – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify this Summer Internship project report **"Http and Https Password Capture Using Wireshark"** is the Bonafide work of **ELAVARASAN S (810023205307)** has satisfactory completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2025-2026.**

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on_____.

1. INTRODUCTION

Cyber Security has become one of the most critical domains in the modern IT industry, as organizations increasingly rely on networked systems to store, transmit, and process sensitive information. Every day, massive volumes of data — including login credentials, financial details, and personal information — travel across networks in the form of small units called packets. If this data is not properly protected at the transport layer, it becomes an easy target for attackers using freely available packet-sniffing tools such as Wireshark, tcpdump, or Ettercap. The consequences of such exposure range from simple privacy violations to large-scale financial fraud and identity theft.

Historically, the World Wide Web was built on the Hypertext Transfer Protocol (HTTP), a simple, stateless, plaintext protocol designed for exchanging documents. HTTP was never designed with confidentiality in mind — every byte exchanged between a browser and a server, including form fields such as usernames and passwords, is transmitted exactly as typed, with no encryption whatsoever. As the web evolved from static documents into full-fledged platforms for banking, shopping, healthcare, and social interaction, this lack of confidentiality became a serious liability. HTTPS (HTTP Secure) was introduced to address this gap by wrapping HTTP inside the Transport Layer Security (TLS) protocol, which encrypts the entire communication channel between the client and the server.

This internship, undertaken at Astonish InfoTech, Trichy, focused on understanding the practical difference between HTTP and HTTPS protocols from a security standpoint. A demo banking login page ("SecureBank Demo") was built and hosted locally using XAMPP on Kali Linux, and Wireshark was used to capture and analyze network traffic in both HTTP (plaintext) and HTTPS (encrypted, TLS-secured) modes. The project demonstrates, hands-on, why HTTPS is essential for any application that transmits sensitive user data, and walks through the complete process of identifying the vulnerability, exploiting it in a controlled lab setting, and then remediating it through proper server-side configuration.

The internship was carried out entirely within an isolated virtual lab environment using Kali Linux, so that no real user data or production system was involved at any stage. This approach reflects standard practice in the cyber security industry, where vulnerabilities are first demonstrated safely in a lab before any recommendation is made to a live system. The skills exercised during this project — web server administration, protocol analysis, and secure configuration — are foundational to roles such as Security Analyst, Penetration Tester, and SOC (Security Operations Center) Analyst.

1.1 Objective of the Internship

- To understand the fundamentals of network protocols, particularly HTTP and HTTPS, and how data travels across the OSI and TCP/IP models.
- To gain hands-on experience with Kali Linux, XAMPP (Apache, MySQL, PHP), and Wireshark in a controlled lab environment.
- To set up a local web server and simulate a real-world login form for a banking application.
- To capture and analyze network packets to observe how credentials travel in plaintext over HTTP.
- To configure SSL/TLS (mod_ssl) on Apache and enable HTTPS on the same application.
- To compare captured packets under HTTP and HTTPS and demonstrate the security benefit of encryption.
- To understand certificate-based trust, the TLS handshake, and why browsers display warnings for self-signed certificates.
- To document the entire process professionally, following industry-standard reporting practices used in real penetration-testing engagements.

1.2 Scope of the Project

The scope of this project is intentionally limited to a local, single-machine lab environment so that the demonstration remains safe, reproducible, and legal. The project does not target any real banking website, third-party service, or production system; the "SecureBank Demo" application is an entirely self-contained mock-up created solely for educational purposes. The scope covers:

- Setting up a local Apache/MySQL/PHP (XAMPP) stack on Kali Linux.
- Designing and deploying a simple HTML/PHP login form that mimics a banking customer portal.
- Capturing and analyzing traffic exclusively on the loopback (localhost) network interface.
- Enabling and verifying TLS encryption using Apache's built-in mod_ssl module with a self-signed certificate.
- Comparing and documenting the observable difference between unencrypted and encrypted traffic.

1.3 Importance of Network Security in Modern Applications

Modern web applications, especially those in the banking, healthcare, and e-commerce sectors, handle highly sensitive information on every single page load. A single unencrypted login request can expose a username and password to anyone positioned on the same network path — a co-worker on the same office Wi-Fi, a malicious actor on a public hotspot, or even a compromised network device. Because of this risk, HTTPS has moved from being a "nice-to-have" feature to being an absolute baseline requirement. Modern browsers such as Chrome and Firefox now actively flag plain HTTP sites as "Not Secure," and search engines penalize non-HTTPS sites in their rankings, reflecting how seriously the industry now treats transport-layer encryption.

1.4 Tools and Technologies Used

The following tools and technologies formed the technical foundation of this internship project. Each was chosen for being free, open-source, and widely used in the cyber security industry, so that the skills gained would be directly transferable beyond this specific exercise.

Tool / Technology	Purpose in this Project
Kali Linux	Security-focused Linux distribution used as the base operating system for the entire lab.
XAMPP	Bundled Apache, MySQL, and PHP stack used to host the demo login application.
Apache mod_ssl	Apache module used to enable SSL/TLS and serve the application over HTTPS.
Wireshark	Network protocol analyzer used to capture and inspect HTTP/HTTPS traffic.
HTML / CSS	Used to design the SecureBank Demo customer login interface.
PHP	Server-side language used in login.php to receive and process form submissions.
Linux Terminal (Bash)	Used for all setup, configuration, and verification commands (chmod, find, grep, ss, curl, etc.).
OpenSSL (via mod_ssl)	Underlying cryptographic library providing the TLS 1.3 implementation used by Apache.

2. COMPANY PROFILE

Astonish InfoTech [P] Ltd

Address:

G-7, Technology Park (BUTP), Bharathidasan University,
Kajamalai Campus, near Law College,
Tiruchirappalli – 620023, Tamil Nadu.
Contact: +91 99949 54031 | Email: admin@astonishinfotech.com
Website: https://www.astonishinfotech.com/

Astonish InfoTech is a Trichy-based software and web solutions company that develops websites, custom software, ERP and CRM platforms, mobile applications, and cloud- and IoT-based systems for businesses and educational institutions. The company also works in emerging technical domains such as artificial intelligence, embedded systems, IoT, VLSI support, and cyber security, offering services like secure coding practices, SSL implementation, and basic data-protection consulting.

The company follows a structured four-stage development process — requirement discussion, design, development, and deployment — and offers a suite of ready-to-customize products including School ERP, billing software, hospital management systems, e-commerce platforms, CRM software, and inventory management systems.

2.1 Services Offered

- Web Development – responsive business websites and dynamic web portals.
- Software Development – custom billing, inventory, accounts, and automation software.
- Mobile App Development – Android and mobile-friendly business applications.
- ERP & CRM Solutions – school/college ERP, lead management, and sales tracking systems.
- Cyber Security – secure coding, SSL configuration, and data-protection practices.
- Cloud & IoT Solutions – cloud deployment, hosting, and smart monitoring systems.

2.2 Technical Domains

In addition to its core web and software services, Astonish InfoTech is active in several advanced technical domains that reflect the changing needs of its client base:

- Artificial Intelligence & Machine Learning – prediction systems, intelligent dashboards, and automation tools.
- Embedded Systems – microcontroller-based applications and hardware integrations.
- Internet of Things (IoT) – smart monitoring, sensor integration, and device control systems.
- VLSI Support – digital design and simulation support for academic and industry projects.
- Cloud Solutions – cloud deployment, backup, hosting, and scalable web infrastructure.

2.3 Development Process

The company follows a structured, four-stage development process for every client engagement, which was also reflected in the mentoring style used during this internship:

- Discuss – requirement analysis and understanding client/project goals.
- Design – UI/UX planning and system architecture.
- Develop – coding, integration, and iterative testing.
- Deploy – launch, monitoring, and post-deployment support.

2.4 Products and Domain Focus

Astonish InfoTech also maintains a set of ready-to-customize software products, including School ERP, billing software, hospital management systems, e-commerce platforms, CRM software, and inventory management systems. Its project portfolio spans education software, retail business systems, industrial automation (IoT-based monitoring), office management dashboards, digital marketing websites, and secure database-driven applications — giving interns exposure to a wide cross-section of real-world software domains, including the cyber security concepts applied in this project.

3. DAILY WORK REPORT

Company Name: Astonish Infotech

Internship Duration: One Month

Domain: Cybersecurity

Topic/Activity:

Day 1

Attended the internship orientation and learned about the company, internship objectives, rules, daily work schedule and the importance of cybersecurity.

Day 2

Studied the fundamentals of cybersecurity, including cyber threats, vulnerabilities, risks, attacks and security controls.

Day 3

Learned about different types of cyberattacks such as malware, viruses, worms, Trojans, ransomware, phishing and social engineering attacks.

Day 4

Studied the CIA Triad principles—Confidentiality, Integrity and Availability—and their importance in protecting information.

Day 5

Learned cybersecurity best practices such as creating strong passwords, enabling multi-factor authentication, updating software and avoiding suspicious links.

Day 6

Studied networking fundamentals, including LAN, WAN, network topologies, routers, switches, hubs, firewalls and access points.

Day 7

Learned the seven layers of the OSI model and the layers of the TCP/IP model, along with their functions and protocols.

Day 8

Studied IPv4 addressing, public and private IP addresses, subnet masks, default gateways, MAC addresses and DNS servers.

Day 9

Learned about common networking protocols such as HTTP, HTTPS, FTP, SSH, DNS, DHCP, TCP, UDP, SMTP and ICMP.

Day 10

Practised basic networking commands such as ping , ipconfig , ifconfig , traceroute , netstat and nslookup for troubleshooting network issues.

Day 11

Learned the fundamentals of the Linux operating system, Linux distributions, directory structure and its applications in cybersecurity.

Day 12

Practised Linux commands such as pwd , ls , cd , mkdir , touch , cat , cp , mv , rm and clear .

Day 13

Studied Linux users, groups, file ownership and permissions using commands such as chmod , chown , sudo and whoami .

Day 14

Practised Linux process and service management commands such as ps , top , kill , grep , systemctl and service .

Day 15

Practised Linux networking commands such as ip addr , ping , ss , netstat , traceroute , hostname and curl .

Day 16

Studied ethical hacking concepts, legal restrictions, hacker classifications and the responsibilities of an ethical hacker.

Day 17

Learned the different phases of ethical hacking, including reconnaissance, scanning, gaining access, maintaining access and reporting.

Day 18

Studied passive and active information-gathering techniques used to collect publicly available information during authorised security testing.

Day 19

Practised basic Nmap commands to identify active devices, open ports, operating systems and running network services in a controlled lab.

Day 20

Used Wireshark to capture and analyse network packets, IP addresses, protocols, source and destination details in a laboratory environment.

Day 21

Studied vulnerabilities, vulnerability assessment, security weaknesses, software patching and the importance of regularly updating systems.

Day 22

Learned web application fundamentals, including clients, web servers, browsers, URLs, domains, cookies, sessions, requests and responses.

Day 23

Studied HTTP and HTTPS methods such as GET and POST, HTTP headers, status codes and secure communication using SSL/TLS.

Day 24

Learned about the OWASP Top 10 and common web application security risks such as injection, broken access control and security misconfiguration.

Day 25

Studied SQL Injection attacks, their causes, possible effects and prevention methods such as input validation and parameterised queries.

Day 26

Learned about Cross-Site Scripting attacks, including reflected, stored and DOM-based XSS, along with basic prevention techniques.

Day 27

Studied authentication and session security, including weak passwords, brute-force attacks, session hijacking and secure login practices.

Day 28

Learned the basic features of Burp Suite and practised configuring the proxy to inspect HTTP requests and responses in a safe lab.

Day 29

Performed basic network and web application security testing activities using Nmap, Wireshark and Burp Suite in an authorised environment.

Day 30

Reviewed all cybersecurity topics, organised the completed activities and prepared the final internship report and documentation.

4. BLOCK DIAGRAM

The block diagram below illustrates the overall architecture of the project. A single Apache server (via XAMPP) hosts the same login application in two modes — HTTP on port 80 and HTTPS on port 443. Wireshark listens on the loopback interface to capture the traffic exchanged between the browser and the server in both cases.

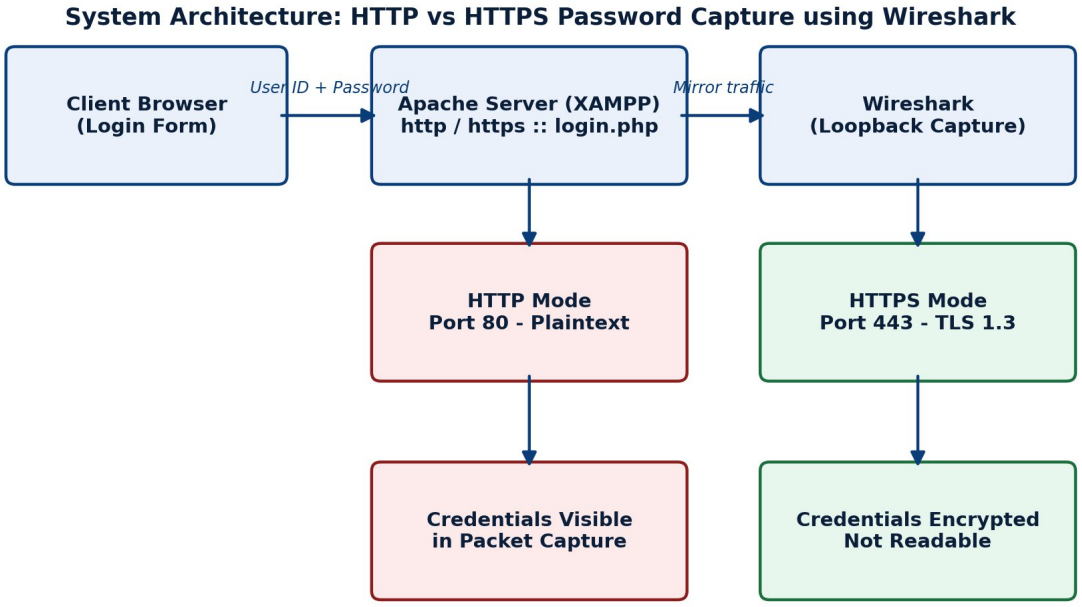


Fig 4.1: System Architecture – HTTP vs HTTPS Password Capture

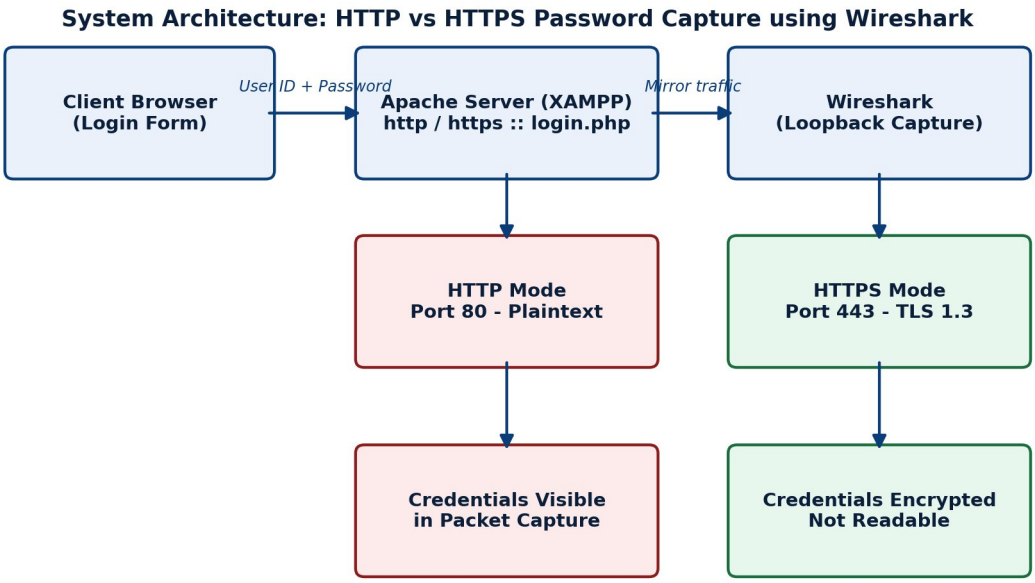


Fig 4.1: System Architecture – HTTP vs HTTPS Password Cap

4.1 Component Description

- Client Browser – renders the SecureBank Demo login form and sends the entered User ID and Password to the server when the Login button is clicked.
- Apache Server (XAMPP) – receives the request on either port 80 (HTTP) or port 443 (HTTPS) and passes it to login.php for processing.
- login.php – reads the submitted form fields on the server side and returns a generic confirmation page; it does not echo the credentials back to the client.
- Wireshark (Loopback Capture) – passively observes all traffic on the lo interface, without interfering with the client-server exchange, and displays it for analysis.
- mod_ssl / TLS Layer – present only in the HTTPS path; wraps the HTTP request and response inside an encrypted TLS 1.3 tunnel before it reaches the network.

4.2 Data Flow Summary

In the HTTP path, the request travels from the browser to Apache and is mirrored to Wireshark as plain, human-readable text — the User ID and Password fields can be read directly from the packet bytes. In the HTTPS path, the same logical request is first encrypted by the TLS layer before it ever reaches the network; Wireshark still captures every byte on the wire, but those bytes are now cipher-text, so the credentials cannot be recovered without the private decryption key. This single architectural change — inserting a TLS layer between the application and the network — is what separates a vulnerable login page from a secure one.

5. HTTP AND HTTPS PASSWORD CAPTURE USING WIRESHARK

5.1 Existing System

In many legacy or poorly configured web applications, login forms are still served over plain HTTP. HTTP transmits all data, including usernames and passwords, as unencrypted plaintext. Anyone with access to the network path — such as a shared Wi-Fi network, a compromised router, or an attacker performing packet sniffing — can capture these credentials using freely available tools such as Wireshark, without needing to break any encryption. This is a serious and well-known vulnerability, especially for applications like banking portals, where credential theft can lead directly to financial loss.

Even in environments where HTTPS is technically available, misconfiguration is common: developers sometimes forget to redirect HTTP requests to HTTPS, mixed-content pages load some resources over HTTP, or self-signed/expired certificates cause users to click through browser warnings, unknowingly falling back to an insecure connection. In each of these cases, the underlying risk is identical to a site that never implemented HTTPS at all — an attacker sniffing the network can recover the transmitted credentials in full.

From an organizational standpoint, the existing-system risk is not limited to the technical exposure itself. Regulatory frameworks such as the RBI's cyber security guidelines for banks and the IT Act, 2000 in India mandate the use of encryption for sensitive data in transit. A banking application that transmits credentials over plain HTTP would be non-compliant with these requirements, in addition to being technically insecure, exposing the organization to both security incidents and regulatory penalties.

5.2 Proposed System

The proposed system demonstrates, in a controlled local environment, exactly how this vulnerability manifests and how it is remedied. A demo banking login page ("SecureBank Demo") is hosted using XAMPP on Kali Linux. The same application is first tested over HTTP, with Wireshark capturing the credentials in plaintext, and then re-tested after enabling SSL/TLS (HTTPS) on the Apache server via mod_ssl. The HTTPS capture is compared against the HTTP capture to show that the TLS handshake and encrypted application data prevent the credentials from being read directly from the packet capture.

The proposed system is designed around three guiding principles: reproducibility (every step is scripted or command-driven so the demonstration can be repeated reliably), observability (Wireshark is used at every stage to provide direct, visual evidence rather than relying on theoretical claims), and safety (the entire exercise is confined to the loopback interface of a single lab machine, with no external network exposure).

5.3 Capabilities

- Hosts a realistic login form and simulates real user credential submission.
- Captures and displays live network traffic using Wireshark on the loopback interface.
- Demonstrates plaintext credential exposure over HTTP with packet-level evidence.
- Demonstrates SSL/TLS configuration on Apache (XAMPP) using mod_ssl and httpd-ssl.conf.
- Confirms HTTPS is active using service checks (ss -ltnp) and endpoint testing (curl -kI).
- Provides a side-by-side comparison of HTTP and HTTPS packet captures for security awareness.
- Can be extended to demonstrate related concepts such as HSTS, certificate validation, and secure cookie flags.

5.4 Advantages of the Proposed Approach

- Low cost — uses only free and open-source tools (Kali Linux, XAMPP, Wireshark, Apache mod_ssl).
- Safe and legal — confined entirely to a local, isolated lab environment.
- Highly visual — the difference between plaintext and encrypted traffic is immediately obvious in Wireshark, making it an effective awareness and training exercise.
- Repeatable — every configuration step is command-line driven, so the entire demonstration can be re-run or automated.

5.5 Limitations

- The demonstration uses a self-signed certificate, so the browser shows a “Not Secure” warning; a production system would require a certificate from a trusted Certificate Authority (CA).
- The capture is performed on the loopback interface for safety; results on a real, shared network would additionally depend on switch/VLAN segmentation and ARP-spoofing resistance.
- The demo application does not implement full authentication logic (password hashing, session management, brute-force protection), as its purpose is limited to illustrating the transport-layer difference between HTTP and HTTPS.

6. PROPOSED METHOD

6.1 Environment Setup

The lab environment was built entirely on Kali Linux, chosen for its pre-installed security and network-analysis toolset. XAMPP for Linux was installed to provide a bundled Apache web server, MySQL database, and PHP interpreter, avoiding the need to configure each component separately. Once installed, the XAMPP control stack was started using sudo /opt/lampp/lampp start, which brought up both the Apache and MySQL services.

The project files (index.html, login.php, README.txt) were placed inside /opt/lampp/htdocs/http_password_capture/, following Apache's default document-root convention so that the application became reachable at http://localhost/http_password_capture/. Folder permissions were then set using sudo chmod -R 755 on the project directory, ensuring the Apache worker process had sufficient read and execute access to serve the files without permission errors.

6.2 HTTP Login and Capture

With XAMPP running in its default configuration, the SecureBank Demo login page was accessed over plain HTTP (port 80). Wireshark was launched and set to capture on the Loopback (lo) interface, since the browser and server were both running on the same machine during this lab exercise. A test User ID and Password were entered into the form and submitted.

The resulting capture clearly showed two significant packets: an HTTP GET request for /http_password_capture/index.html when the page was first loaded, and an HTTP POST request to /http_password_capture/login.php when the form was submitted. Selecting the POST packet in Wireshark and expanding the “HTML Form URL Encoded” section in the packet-detail pane revealed the exact field names and values — username and password — in fully readable plaintext, confirming the vulnerability in a concrete, evidence-based way rather than as a theoretical claim.

6.3 Enabling SSL/TLS (HTTPS)

To secure the application, Apache's SSL module (mod_ssl) was enabled on the XAMPP stack. Before making any changes, the existing httpd.conf and httpd-ssl.conf files were backed up using cp, following standard configuration-management practice so that the original settings could be restored if anything went wrong.

The httpd-ssl.conf file, which contains the SSL virtual host definition, was located under /opt/lampp/etc/extra/ using the find command. Its correct inclusion in the main configuration was then verified using grep -nE "ssl_module|httpd-ssl.conf" httpd.conf, which confirmed that the LoadModule ssl_module directive was present and that the httpd-ssl.conf file was properly Included. Before restarting the server, the configuration was validated using sudo /opt/lampp/bin/httpd -t, which returned "Syntax OK," confirming there were no configuration errors that would prevent Apache from starting.

6.4 HTTPS Login and Capture

After the syntax check passed, XAMPP was restarted using sudo /opt/lampp/lampp restart, which stopped and restarted both Apache and MySQL with the new SSL configuration active. The ss -ltnp | grep ':443' command was then used to confirm that Apache was actively listening for HTTPS connections on port 443.

As an additional verification step, curl -kI https://localhost/http_password_capture/index.html was run from the terminal; the -k flag allowed the self-signed certificate to be accepted for testing purposes, and the response returned HTTP/1.1 200 OK along with server headers confirming Apache, OpenSSL, and PHP were all active over the encrypted connection. The login page was then accessed via https://localhost/http_password_capture/index.html in the browser (which displayed a "Not Secure" warning due to the self-signed certificate, as expected in a lab setting), and the same test credentials were submitted.

Wireshark's capture on this run looked fundamentally different from the HTTP capture. Instead of a readable GET/POST exchange, the packet list showed a TLS 1.3 handshake sequence — Server Hello, Change Cipher Spec, and Finished — followed by Encrypted Application Data frames. Selecting the frame corresponding to the login submission showed only cipher-text at the byte level; no combination of "Follow HTTP Stream" or form-decoding features in Wireshark could recover the username or password, because the data was never transmitted in plaintext on the wire at any point.

6.5 Testing and Verification Approach

Each stage of the proposed method was verified independently before moving to the next, following a systematic testing approach:

- Service-level testing — confirming Apache and MySQL were running using the XAMPP status output after every start/restart command.
- Configuration-level testing — validating httpd.conf and httpd-ssl.conf syntax with httpd -t before every restart, to catch errors before they caused downtime.
- Network-level testing — using ss -ltnp to confirm the correct ports (80 and 443) were open and listening.
- Application-level testing — using curl to verify HTTP response codes independently of the browser.
- Packet-level testing — using Wireshark as the final, ground-truth verification of what data actually crosses the network in each mode.

6.6 Challenges Faced and Solutions

Several practical challenges came up during implementation, each of which added to the overall learning experience:

- Challenge: An early attempt to back up the configuration using sudo /opt/lampp/etc/httpd.conf directly failed with a “command not found” error, because the path was mistakenly treated as an executable rather than a file argument to cp. Solution: The correct cp source-destination syntax was used instead (sudo cp httpd.conf httpd.conf.backup).
- Challenge: Locating the exact path of the SSL configuration file was not obvious, since XAMPP’s directory layout differs from a standard Linux Apache installation. Solution: The find command (sudo find /opt/lampp/etc -name "httpd-ssl.conf") was used to search the filesystem directly rather than guessing the path.
- Challenge: After the first SSL configuration attempt, it was not immediately clear whether the ssl_module was actually being loaded. Solution: grep -nE "ssl_module|httpd-ssl.conf" httpd.conf was used to directly confirm both the LoadModule directive and the Include statement were present and correctly referenced.
- Challenge: The browser displayed a “Not Secure” warning even after HTTPS was correctly enabled. Solution: This was identified as expected behaviour for a self-signed certificate (not a configuration error), since the certificate is not issued by a browser-trusted Certificate Authority — a distinction documented in Chapter 5.5 (Limitations).
- Challenge: Confirming that Wireshark was capturing the right traffic required selecting the correct interface, since local browser-to-localhost traffic does not appear on a regular network adapter. Solution.

7. ALGORITHMS / METHODOLOGY

7.1 User Authentication Flow

- Step 1: User opens the SecureBank Demo login page in the browser.
- Step 2: User enters the User ID and Password and clicks the Login button.
- Step 3: The browser sends the form data to login.php using an HTTP POST request.
- Step 4: login.php processes the request and returns a “Login Request Submitted” confirmation page.

7.2 Packet Capture Process

- Step 1: Launch Wireshark and select the appropriate capture interface (Loopback: lo for localhost testing).
- Step 2: Start the live capture before submitting the login form.
- Step 3: Trigger the login action in the browser to generate network traffic.
- Step 4: Stop the capture and apply a display filter (e.g., http) to isolate relevant packets.
- Step 5: Inspect the packet details pane to view headers, form data, or TLS records.

7.3 SSL/TLS Configuration Logic

- Step 1: Locate the SSL configuration file (httpd-ssl.conf) using find.
- Step 2: Back up httpd.conf and httpd-ssl.conf before making changes.
- Step 3: Confirm ssl_module is loaded and httpd-ssl.conf is included, using grep.
- Step 4: Validate the configuration syntax using httpd -t.
- Step 5: Restart the XAMPP stack and confirm port 443 is listening using ss -ltnp.

7.4 HTTP vs HTTPS Comparison Logic

- Capture and record packets for the same login action once over HTTP and once over HTTPS.
- Compare the two captures: HTTP shows the form fields and values in plaintext; HTTPS shows only the encrypted handshake and application data.
- Conclude that HTTPS protects credentials in transit, while HTTP does not.

7.5 Data Storage and Retrieval (Capture Files)

- Step 1: Save each Wireshark capture as a .pcapng file immediately after stopping it, to preserve the evidence for later review.
- Step 2: Name capture files clearly (e.g., http_capture.pcapng, https_capture.pcapng) to avoid confusion during comparison.

- Step 3: Reopen saved captures using File > Open in Wireshark whenever the results need to be re-verified or included in documentation.
- Step 4: Export specific packets or objects (e.g., HTTP > Export Objects) if a particular request needs to be studied in isolation.

7.6 Screenshot and Evidence Collection

- Step 1: Capture a screenshot at each meaningful checkpoint — environment setup, HTTP test, SSL configuration, and HTTPS test.
- Step 2: Highlight or annotate the key evidence in each screenshot (e.g., the plaintext form fields in the HTTP capture).
- Step 3: Arrange the screenshots in chronological, logical order to build a clear before-and-after narrative for the report.

8. CODE IMPLEMENTATION

The project consists of two core files placed inside /opt/lampp/htdocs/http_password_capture/: index.html, the customer login form, and login.php, which receives the submitted credentials. A representative, simplified version of the structure used is shown below, along with an explanation of each part.

8.1 index.html – Login Form

This file renders the "SecureBank Demo" customer login card seen in the browser. The <form> element uses method="POST", which is what causes the browser to send the User ID and Password as the body of an HTTP POST request rather than appending them to the URL (as a GET request would). This is the exact request that Wireshark intercepts and displays in Chapter 9.

```
<!-- index.html : Customer Login Form (SecureBank Demo) -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>SecureBank Demo Login</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="topbar"><h1>SecureBank Demo</h1></header>
  <main class="login-card">
    <h2>Customer Login</h2>
    <form action="login.php" method="POST">
      <label for="username">User ID</label>
      <input type="text" id="username" name="username"
```

```
            placeholder="Enter User ID" required>

        <label for="password">Password</label>
        <input type="password" id="password" name="password"
            placeholder="Enter Password" required>

        <button type="submit">Login</button>
    </form>
  </main>
</body>
</html>
```

8.2 login.php – Form Handler

This server-side script reads the incoming $_POST['username'] and $_POST['password'] values once the form is submitted. It intentionally does not print the credentials back onto the confirmation page — this design choice ensures that the only place the raw values can be observed by an outside party is on the network itself, which is precisely the behavior this project sets out to demonstrate.

```
<?php
// login.php : receives POST data submitted from index.html
// NOTE: Demo only — no database, hashing, or session logic included,
// since the purpose of this file is to illustrate transport-layer
// exposure, not to implement production authentication.

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = $_POST['username'] ?? '';
    $password = $_POST['password'] ?? '';

    // In a real application: validate input, hash-compare the
    // password against a stored hash, and start a secure session.
    // For this demo, the values are intentionally NOT echoed back,
    // so the only place they can be observed is the network capture.
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Login Result</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="result-card">
    <h2 class="success">Login Request Submitted</h2>
    <p>Your login request has been processed.</p>
    <p>This page does not display username or password.</p>
    <a class="btn" href="index.html">Back to Login</a>
```

```
  </main>
</body>
</html>
```

8.3 How the Form Data Appears on the Wire

When the form in index.html is submitted, the browser encodes the two fields as application/x-www-form-urlencoded data and places them in the POST request body, roughly in the form username=Elavarasan&password=12345678. Over HTTP, this exact string appears untouched inside the TCP payload and is what Wireshark displays under “HTML Form URL Encoded” in the packet-detail pane. Over HTTPS, the same string is generated by the browser, but it is encrypted by the TLS layer before being handed to the operating system's network stack, so Wireshark only ever sees the resulting cipher-text.

8.4 Server-Side Commands Used

Key terminal commands used to set up, secure, and verify the environment during the project:

- sudo /opt/lampp/lampp start – starts Apache and MySQL services.
- sudo chmod -R 755 /opt/lampp/htdocs/http_password_capture – sets folder permissions.
- sudo find /opt/lampp/etc -name "httpd-ssl.conf" – locates the SSL configuration file.
- sudo cp httpd.conf httpd.conf.backup – backs up the configuration before editing.
- sudo cp httpd-ssl.conf httpd-ssl.conf.backup – backs up the SSL configuration before editing.
- grep -nE "ssl_module|httpd-ssl.conf" httpd.conf – verifies SSL module inclusion.
- sudo /opt/lampp/bin/httpd -t – checks Apache configuration syntax before restarting.
- sudo /opt/lampp/lampp restart – restarts XAMPP with SSL enabled.
- ss -ltnp | grep ':443' – confirms Apache is listening for HTTPS connections.
- curl -kI https://localhost/http_password_capture/index.html – tests the HTTPS endpoint and confirms a 200 OK response.

8.5 Design Notes

The application was deliberately kept minimal — two files, no database, no external dependencies — so that the network-level behaviour being studied would not be obscured by unrelated application complexity. This mirrors a common practice in security testing,

where a minimal, reproducible “proof of concept” is preferred over a fully featured application, because it isolates the exact vulnerability being demonstrated.

9. SAMPLE OUTPUT

This section presents the project execution in sequence, from environment setup through to the HTTP and HTTPS packet captures in Wireshark. Each figure is accompanied by a short caption describing the checkpoint it represents, so that the progression from an insecure to a secure configuration can be followed step by step.

9.1 Project Environment Setup

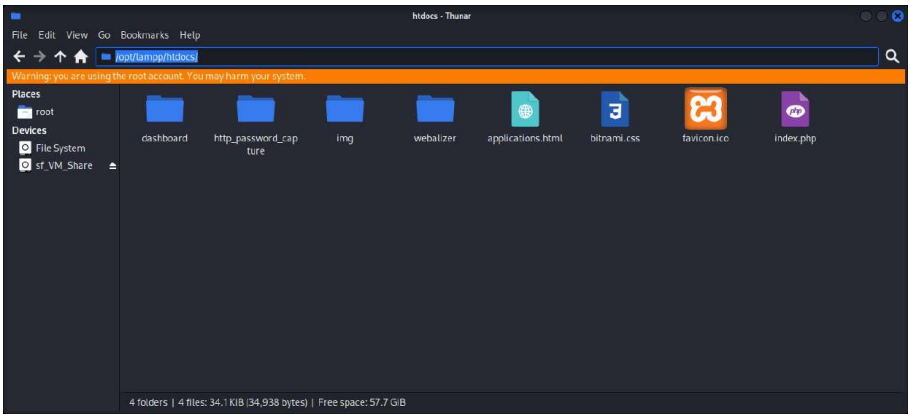


Fig 9.1: XAMPP htdocs directory showing the http_password_capture project folder

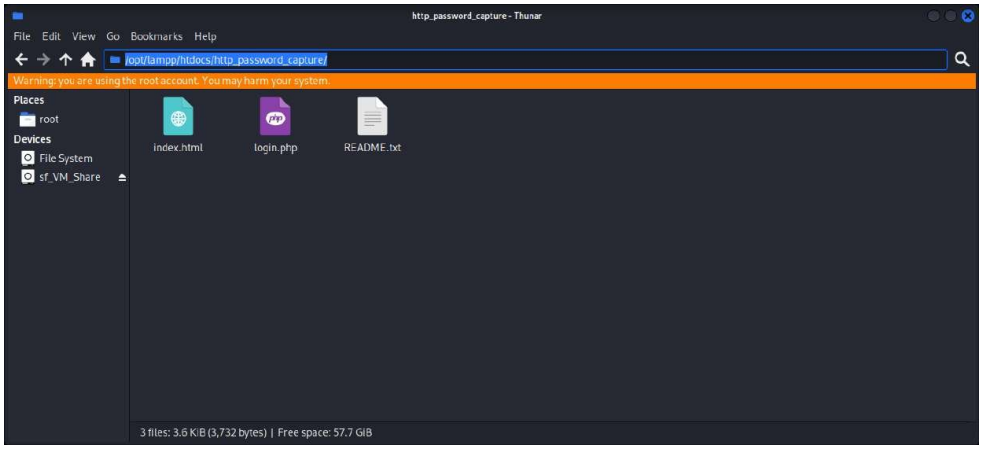


Fig 9.2: Project folder contents – index.html, login.php, and README.txt

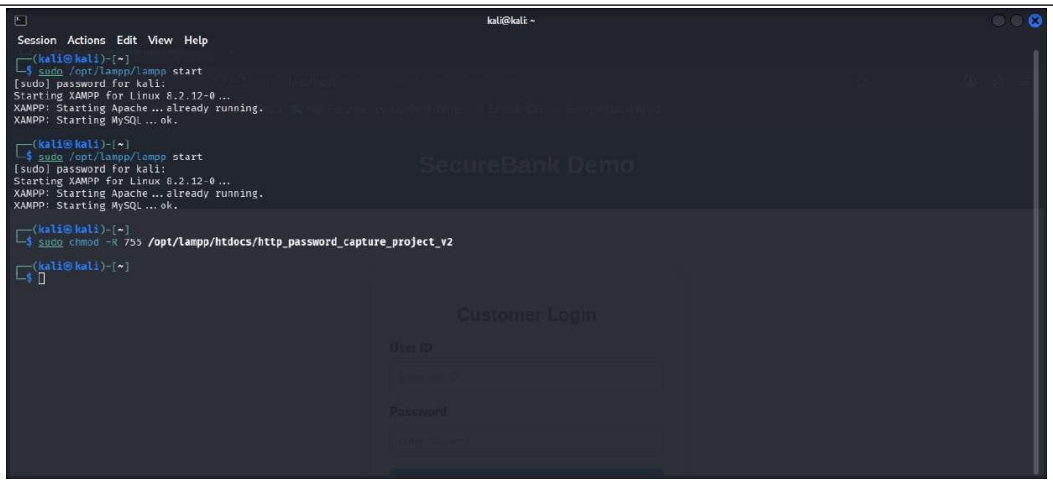


Fig 9.3: Starting XAMPP (Apache & MySQL) and setting folder permissions via chmod

9.2 Login and Capture over HTTP

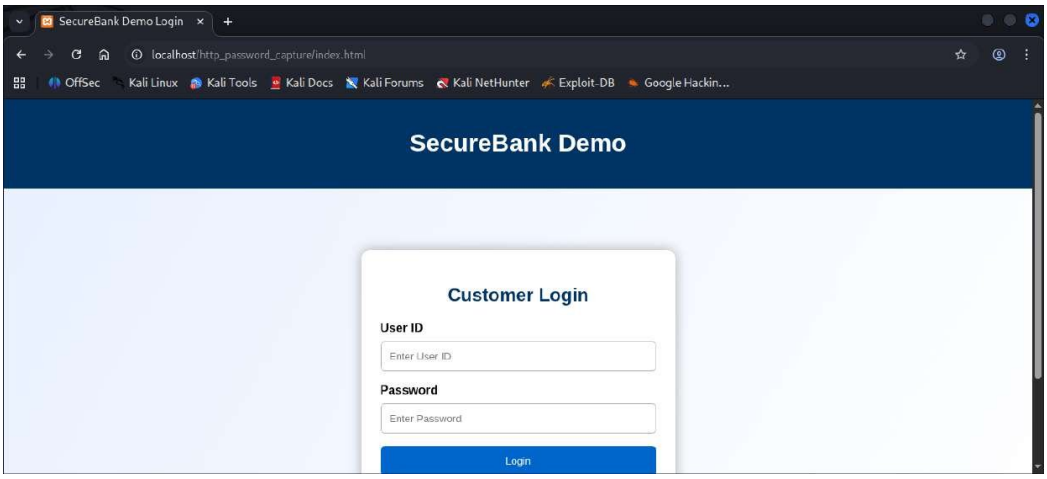


Fig 9.4: SecureBank Demo login page served over HTTP

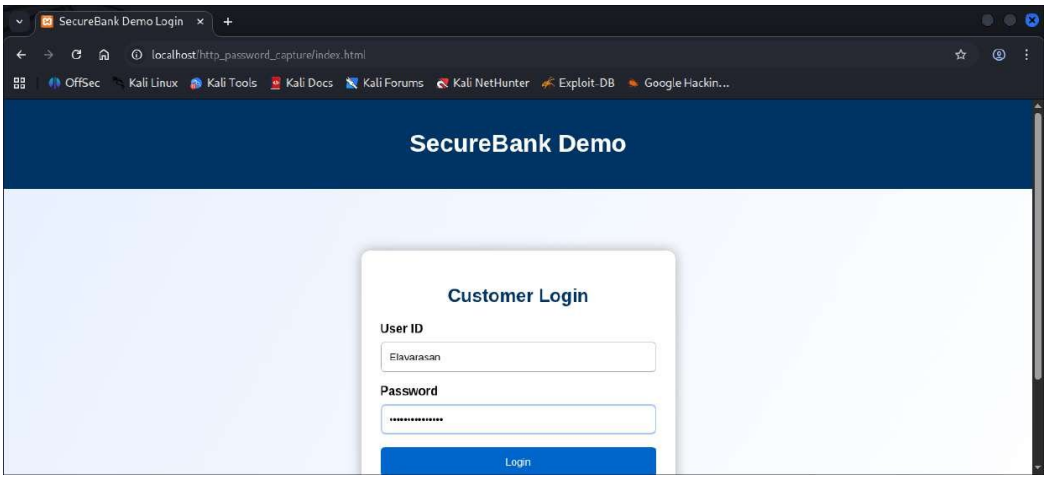


Fig 9.5: Login form filled with test credentials over HTTP

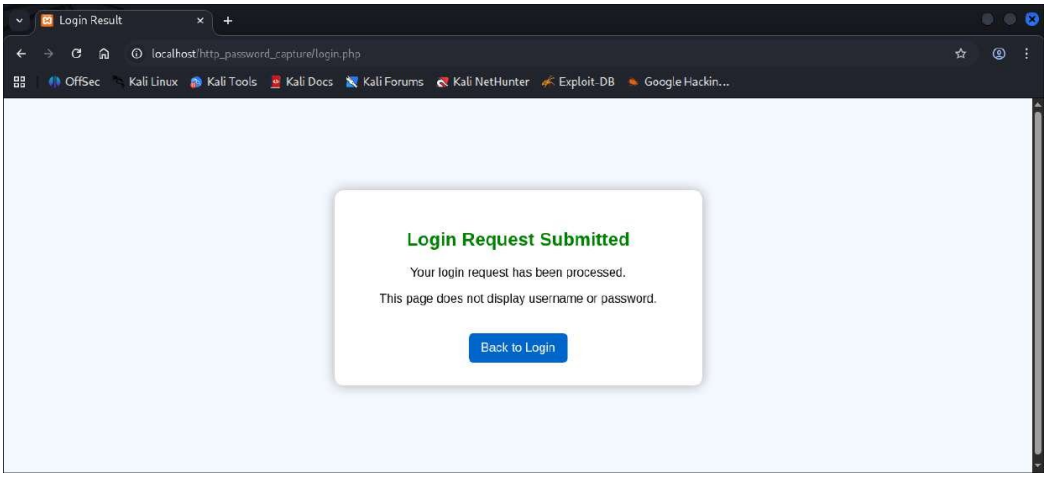


Fig 9.6: "Login Request Submitted" confirmation page (HTTP)

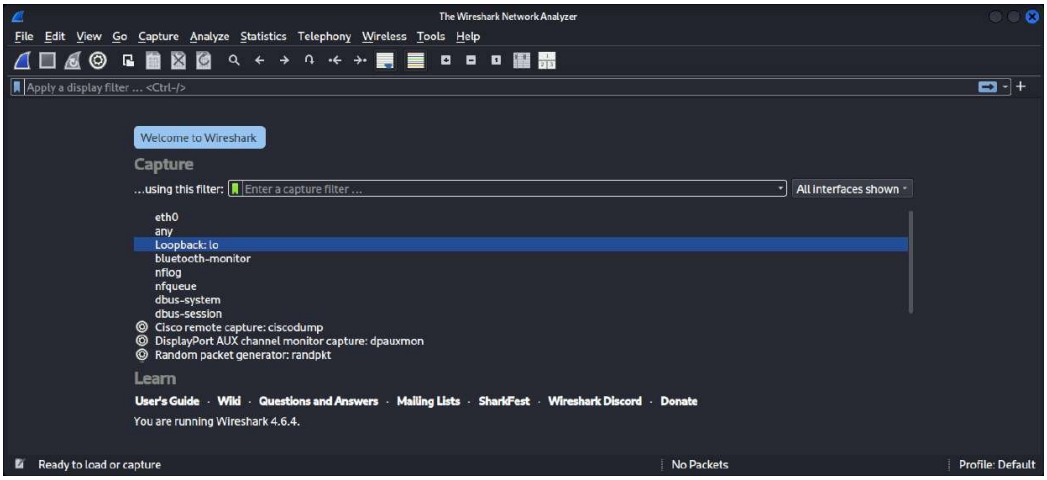


Fig 9.7: Wireshark interface selection screen – Loopback: lo

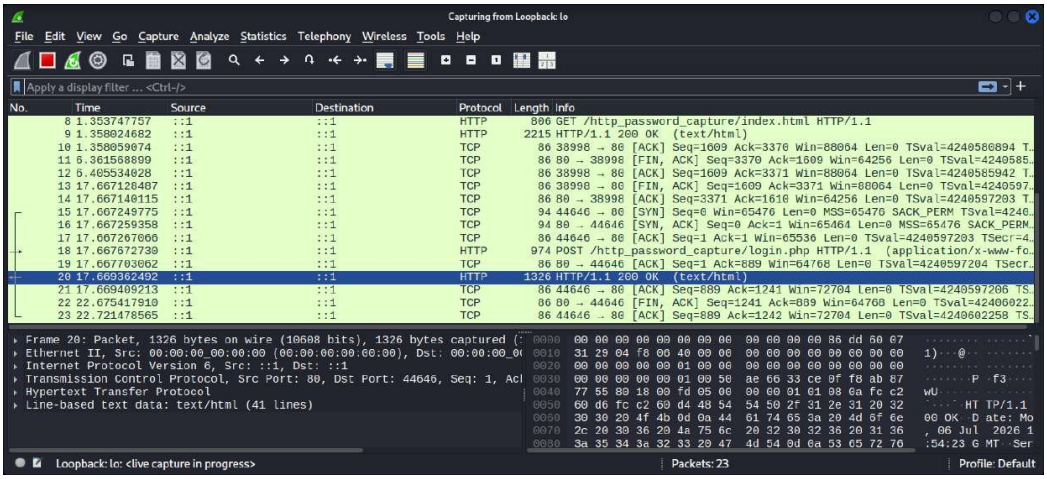


Fig 9.8: Wireshark capture showing the HTTP GET and POST packets, with form data visible in plaintext

9.3 Enabling SSL/TLS (HTTPS) on Apache

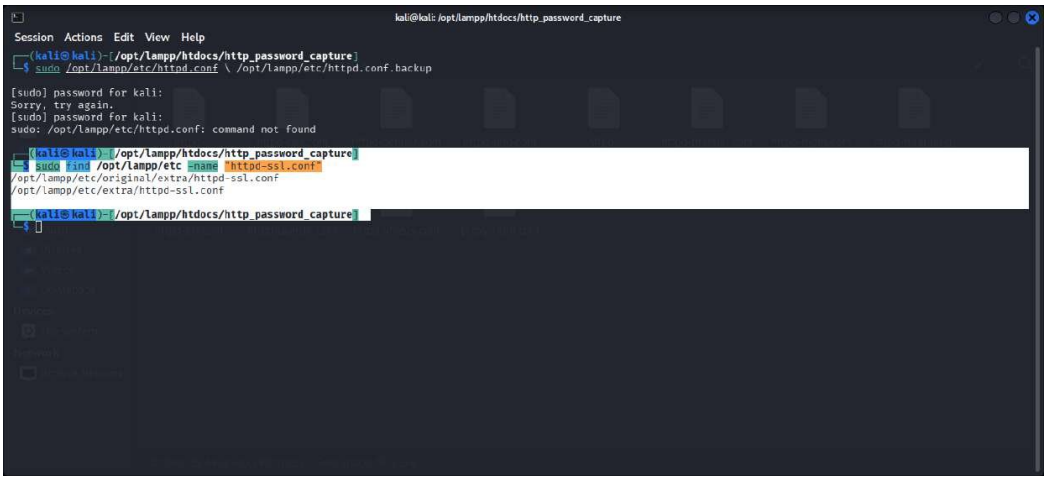


Fig 9.9: Locating the httpd-ssl.conf file using the find command

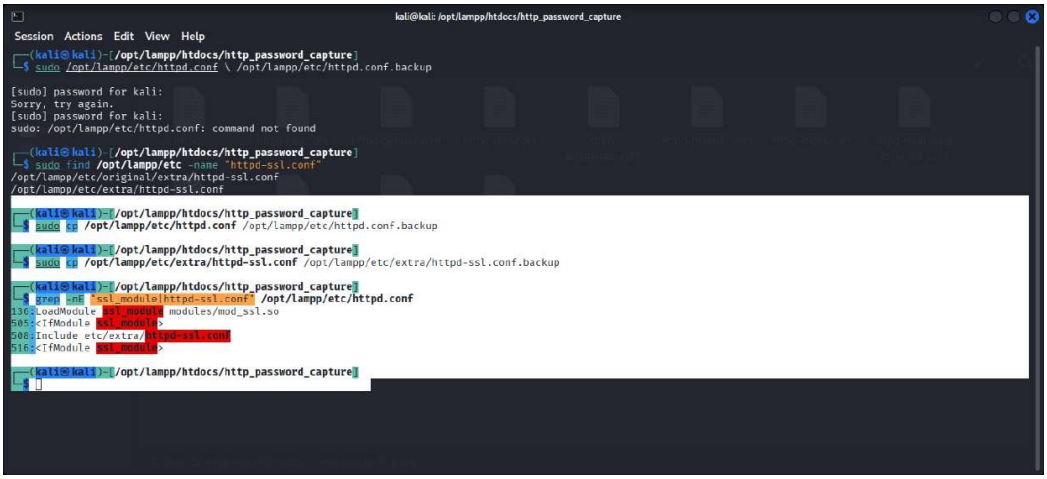


Fig 9.10: Backing up configuration files and verifying SSL module inclusion using grep

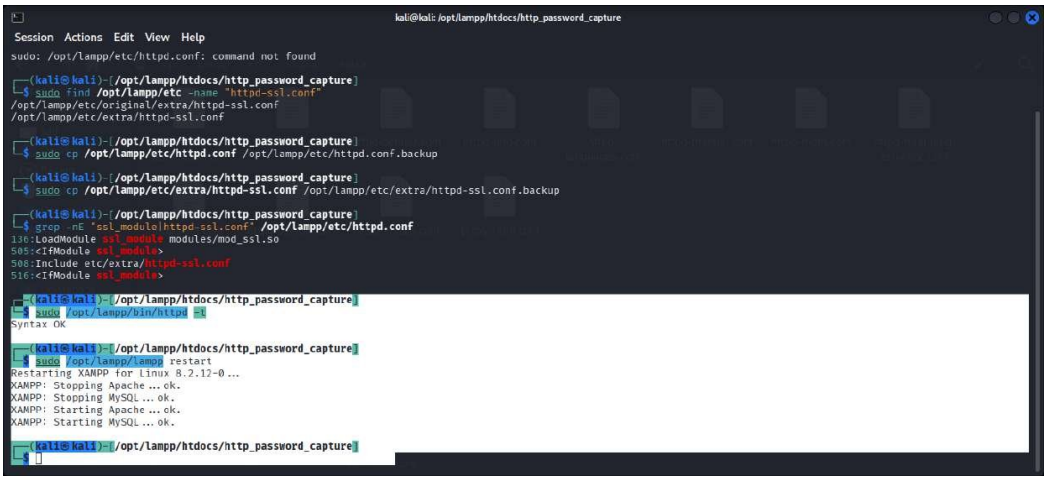


Fig 9.11: Apache configuration syntax check (httpd -t) and XAMPP restart with SSL enabled

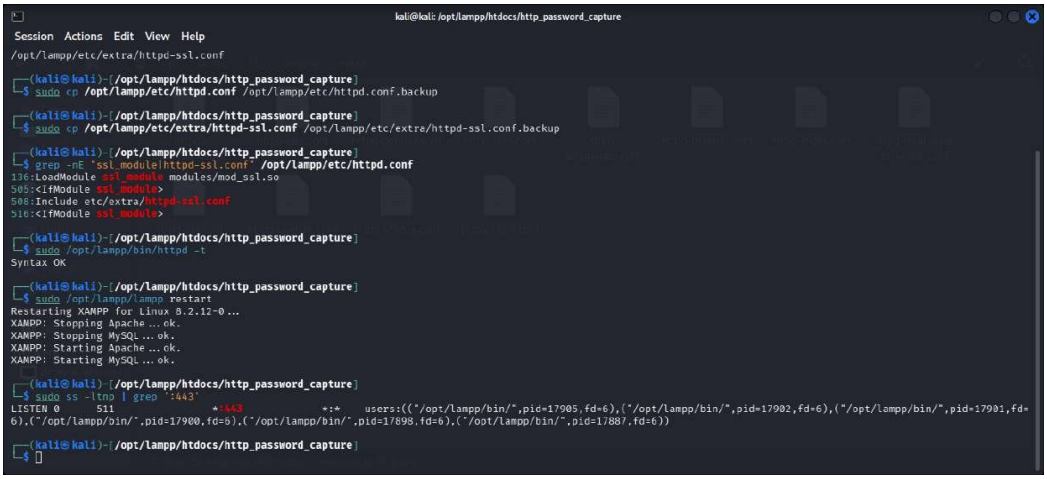


Fig 9.12: Verifying Apache is listening on port 443 using ss -ltnp



Fig 9.13: Testing the HTTPS endpoint using curl -kI – 200 OK response confirms SSL is active

9.4 Login and Capture over HTTPS

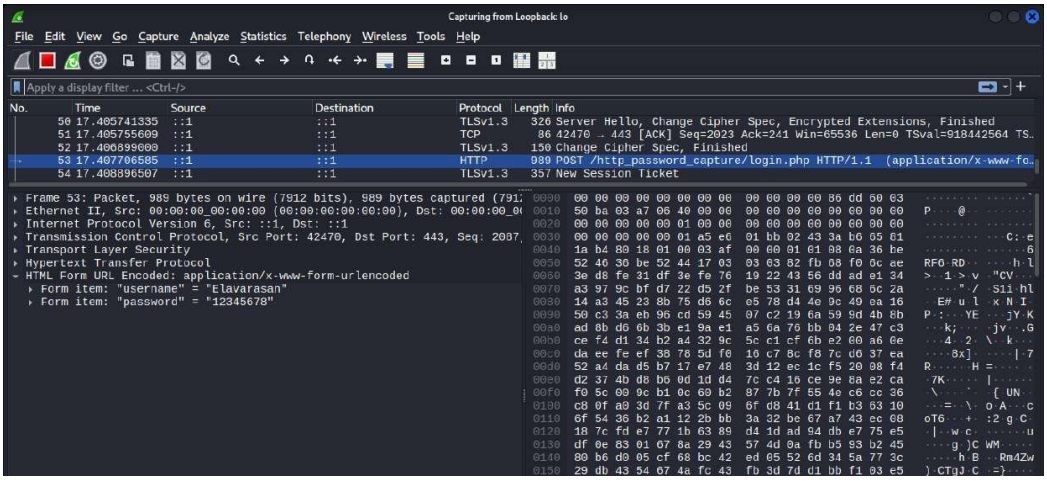


Fig 9.14: Wireshark capture over HTTPS – TLS 1.3 handshake and encrypted application data

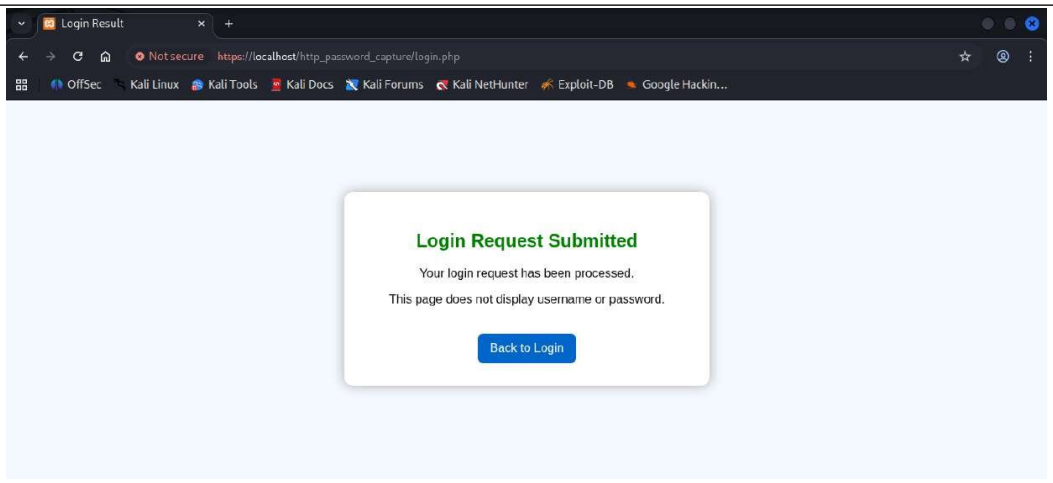


Fig 9.15: "Login Request Submitted" confirmation page served over HTTPS (secure connection)

10. ANALYSIS

The table below summarizes the key differences observed between the HTTP and HTTPS packet captures taken during this project.

Parameter	HTTP (Port 80)	HTTPS (Port 443)
Data format on the wire	Plaintext	Encrypted (TLS 1.3)
Username/Password visibility	Fully visible in packet capture	Not visible; only encrypted application data seen
Protocol observed in Wireshark	HTTP GET / POST	TLS handshake, Change Cipher Spec, Encrypted Application Data
Server verification (curl -kI)	Not applicable	200 OK over TLS confirms SSL is active
Certificate involved	None	Self-signed certificate (browser shows "Not Secure" warning)
Risk level for credential theft	High – exploitable with basic packet sniffing	Low – requires breaking TLS encryption

The comparison clearly demonstrates that HTTP is fundamentally unsafe for transmitting sensitive data such as login credentials, since anyone capturing traffic on the same network segment can read the username and password directly. Enabling HTTPS through proper SSL/TLS configuration on the Apache server closes this gap, ensuring the same data is unreadable to a network-level observer. This reinforces why HTTPS is a mandatory requirement, not an optional enhancement, for any application handling authentication or financial data.

10.1 Security Implications

The HTTP capture in this project stands in for a realistic and still-common failure: a login form that appears fully functional to the end user, giving no visual indication that it is insecure, while silently exposing every submitted password to anyone capturing traffic on the same segment. This is significant because the average user has no way of detecting the risk without specifically checking for the padlock icon and "https://" prefix in the address bar. From an organizational risk perspective, a single unencrypted login endpoint can be enough to compromise an entire user base if traffic is intercepted at a shared network chokepoint such as a public Wi-Fi access point or a compromised office router.

The HTTPS capture demonstrates the corresponding mitigation. Once mod_ssl was enabled and the TLS handshake completed successfully, Wireshark could still see that a login request had occurred (the timing, size, and destination of the packets), but could not recover the actual content of that request. This illustrates an important nuance in network security: TLS protects the confidentiality of data content, but does not hide the fact that communication is taking place — a distinction that is relevant when discussing traffic-analysis attacks, which were outside the scope of this project.

10.2 Performance and Operational Overhead of HTTPS

A common misconception is that enabling HTTPS introduces significant performance overhead. During this project, the curl -kI test against the HTTPS endpoint returned a 200 OK response with negligible perceptible delay compared to the HTTP version, consistent with modern TLS implementations (particularly TLS 1.3, observed in the capture), which are highly optimized and add only a small, often unnoticeable, latency cost for the initial handshake. Given the security benefit gained, this minor overhead is considered an acceptable and standard trade-off across the industry.

10.3 Real-World Relevance

Although this project was carried out on a single local machine for safety and simplicity, the exact same commands, configuration files, and verification steps (locating httpd-ssl.conf, enabling ssl_module, validating with httpd -t, and confirming port 443 with ss -ltnp) apply directly to securing a real Apache-based production server. This makes the exercise directly transferable to real system-administration and security-hardening tasks, rather than being a purely theoretical demonstration.

10.4 Learning Outcomes

- Gained a working understanding of how HTTP and HTTPS differ at the packet level, beyond just the theoretical definition.
- Learned to configure and troubleshoot Apache's SSL module (mod_ssl) on a XAMPP stack, including locating configuration files and validating syntax before deployment.
- Developed practical skills in using Wireshark for live traffic capture, filtering, and packet-level inspection.
- Practiced safe, methodical, command-line-driven system administration — backing up configuration files before editing, and validating each change before moving to the next step.
- Understood, through direct observation, why regulatory and industry standards (RBI guidelines, OWASP recommendations) mandate HTTPS for any application handling sensitive data.
- Improved professional documentation skills by recording each stage of the exercise with clear evidence (terminal output and Wireshark screenshots) suitable for a formal security report.

11. CONCLUSION

This internship provided valuable hands-on experience in network security by practically demonstrating the difference between HTTP and HTTPS. By building a demo banking login page and capturing its traffic in Wireshark under both protocols, the project clearly showed that HTTP exposes user credentials as plaintext, while HTTPS — enabled through proper SSL/TLS configuration on Apache — encrypts the same data, protecting it from network-level interception.

Working through the complete lifecycle of the project — from installing and configuring XAMPP on Kali Linux, to building a realistic login form, to capturing and interpreting

live network traffic, to locating and correctly enabling Apache's SSL module — gave a much deeper, practical understanding of concepts that are often only covered theoretically in coursework. Reading about TLS encryption in a textbook is very different from watching, packet by packet, exactly what an attacker would and would not be able to see on an unprotected versus a protected connection.

The project also reinforced the importance of methodical, evidence-based security work: every claim made in this report — that HTTP exposes credentials, that HTTPS protects them — is backed by an actual Wireshark capture rather than assumption. This discipline of verifying security claims with direct evidence is a core habit for any cyber security professional, whether working as a penetration tester, security analyst, or system administrator.

Overall, this exercise strengthened my understanding of web server configuration, packet analysis, and the real-world importance of secure communication protocols, and it forms a solid foundation for further study in cyber security and ethical hacking, particularly in areas such as network penetration testing and secure application development.

12. FUTURE WORK

While this project successfully demonstrated the core difference between HTTP and HTTPS, several extensions could deepen the analysis and bring it closer to real-world penetration-testing and secure-development practice:

12.1 Advanced Attack Simulation

- Extend the project to demonstrate HTTPS downgrade and man-in-the-middle (MITM) attack scenarios using tools like Ettercap or Bettercap in an isolated lab environment.
- Simulate ARP spoofing on a virtual lab network (with multiple isolated VMs) to show how MITM positioning is achieved in practice, strictly for educational purposes.

12.2 Stronger Security Hardening

- Implement HSTS (HTTP Strict Transport Security) headers to prevent protocol downgrade attacks and force browsers to always use HTTPS for the site.
- Replace the self-signed certificate with one issued by a trusted internal or public Certificate Authority (e.g., Let's Encrypt) to remove browser trust warnings.

- Add secure cookie flags (Secure, HttpOnly, SameSite) once session management is introduced, to further protect authentication tokens.

12.3 Application-Level Improvements

- Add password hashing (bcrypt) and a real database backend to extend the demo into a more complete secure authentication system.
- Introduce basic brute-force protection (rate limiting or account lockout) to complement the transport-layer security already demonstrated.

12.4 Automation and Tooling

- Explore certificate pinning and analyze how it affects packet capture and interception attempts.
- Automate the HTTP vs HTTPS comparison using a Python script with pyshark for repeatable testing and reporting.
- Build a simple dashboard that parses saved .pcapng files and automatically flags any plaintext credentials found, as a reusable internal security-testing utility.

13. REFERENCES

1. Wireshark Official Documentation – https://www.wireshark.org/docs/
2. Apache HTTP Server Documentation – https://httpd.apache.org/docs/
3.Apachemod_sslDocumentation–https://httpd.apache.org/docs/current/mod/mod_ssl.html
4. XAMPP for Linux – https://www.apachefriends.org/
5. Kali Linux Documentation – https://www.kali.org/docs/
6. Astonish InfoTech, Trichy – https://www.astonishinfotech.com/
7. Mozilla Developer Network, "An overview of HTTP" – https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview
8. Mozilla Developer Network, "Transport Layer Security (TLS)" – https://developer.mozilla.org/en-US/docs/Web/Security/Transport_Layer_Security
9. OWASP Foundation, "Transport Layer Protection Cheat Sheet" – https://owasp.org/www-project-cheat-sheets/
10. Reserve Bank of India, Cyber Security Framework for Banks – https://www.rbi.org.in/
11. Information Technology Act, 2000, Government of India.
12. IETF RFC 8446, "The Transport Layer Security (TLS) Protocol Version 1.3" – https://www.rfc-editor.org/rfc/rfc8446

astonish
INFOTECH [P] LTD

Astonish Infotech [p] LTD

G7, Technology Park, Bharathidasan University (BUTP), Tiruchirappalli

This certificate is presented to **Mr. ELAVARASAN. S**, Register number 810023205307 from University College of Engineering (UCE), BIT Campus, Anna University, Trichy of B.Tech. (Information Technology) student have successfully completed the Internship in topic entitled Cyber Security in our Organisation from 01.06.2026 to 30.06.2026.

Astonish Infotech Pvt. Ltd. Trichy-23

AUTHORISED SIGNATURE



Astonish Infotech Pvt. Ltd. Trichy-23

MENTOR SIGNATURE

astonish
INFOTECH [P] LTD

G7, Technology Park,
Bharathidasan University (BUTP),
Kajamalai Campus,
Tiruchirappalli – 620 023

Office: 0431-2331318, 09994954031
Email: admin@astonishinfotech.com
Website: www.astonishinfotech.com

ANNEXURE

To Internship (June-July2026)

1.Name of the Industry with address: Astonish Infotech, G7, Technology Park, Trichy.

2.Name of the Industrial Expert (Joint Supervisor): R. Srikanniga Priyadharshini

3.Qualification of the Industrial Expert (Joint Supervisor): B.Tech-IT (Hons)

4.Designation of the Industrial Expert (Joint Supervisor): Project Manager

5.Title of the Project Work: HTTP PASSWORD CAPTURE USING WIRESHARK

6.Duration of the Internship: 1 Month.

7.Details of the student:

Sl.No	Name of the Student	Registration No	Department
1.	S. Elavarasan	810023205307	IT

Terms and conditions

- The student is permitted to undergo a semester long internship in your organization
- The student shall be instructed to meet the Institute supervisor periodically and to attend the review committee meetings for evaluating the progress.
- The final report of the internship duly signed by the industrial expert/supervisor will be submitted to college.
- Attendance Certificate in the prescribed format signed by the competent authority of the industry will be produced.

Authorised signatory

Astonish Infotech Pvt. Ltd. Trichy-23

ATTENDANCE CERTIFICATE FOR INDUSTRIAL TRAINING / INTERNSHIP

(Attendance Certificate to be signed by the competent authority of the industry mentioning the period of Industrial Training / Internship)

The Student Mr./ ~~Ms.~~ S. Elavarasan (Reg.No.) 810023205307,

studying B.Tech / IT programme at VCE, BIT CAMPUS College

in semester VII at Department of B.Tech / IT has attended Industrial

Training / Internship from 01.06.2026 to 30.06.2026 It is certified that he / ~~she has~~

completed / not completed the Industrial Training / Internship in our

Astonish Infotech organization / institution.

Astonish Infotech Pvt. Ltd. Trichy-23

Signature

Name and Designation of the Officer
Seal of the Organization

astonish
INFOTECH [P] LTD

G7, Technology Park,
Bharathidasan University (BUTP),
Kajamalai Campus,
Tiruchirappalli- 620 023

Office: 0431-2331318, 09994954031
Email: admin@astonishinfotech.com
Website: www.astonishinfotech.com

INTERNSHIP COMPLETION CERTIFICATE

TO WHOM IT MAY CONCERN

This is to certify that Mr. Elavarasan S. (Register Number: 810023205307), a B.Tech.(Information Technology) student at University College of Engineering (UCE), BIT Campus, Anna University, Tiruchirappalli, has successfully completed his internship with our organization in the Cyber Security domain.

The internship duration was from 1st June 2026 to 30th June 2026.

During the internship period, he demonstrated good learning ability, dedication, and a positive attitude towards the assigned tasks. He showed strong interest in understanding the practical aspects of Cyber Security, network security, vulnerability assessment, and information protection.

We wish him all the best in his future academic and professional endeavors.

For Astonish Infotech Private Limited

Authorised Signatory

Astonish Infotech Pvt. Ltd Trichy-23